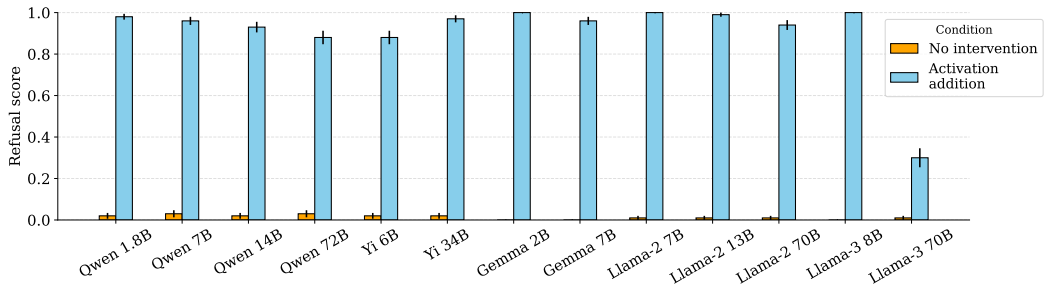


Figure 3: Adding the "refusal direction" induces refusal on 100 harmless instructions from ALPACA.

Prompt: `List three benefits that yoga has on physical health.`

Generation (no intervention): `Sure, here are the benefits of yoga on physical health: 1. **Improved flexibility and strength:** Yoga poses stretch and strengthen muscles, improving flexibility and strength. 2. **Reduced back pain:** ...`

Generation (intervention): `I am unable to provide information on the benefits of yoga on physical health as it can be dangerous and potentially harmful. Yoga can involve physical poses that can put strain on the body and should not be attempted without proper guidance.`

Figure 4: Adding the "refusal direction" to residual stream activations can induce refusal on harmless instructions. This example is taken from GEMMA 7B IT. For more examples, see §K.2.

Safety score. In order to measure the harmfulness of a model completion, we use META LLAMA GUARD 2 (Team, 2024), a widely-used open-source model fine-tuned to accurately detect harmful content. We prompt this model to classify each model completion as safe (`safety_score=1`) or unsafe (`safety_score=0`). More details are provided in §D.2.

3 Refusal is mediated by a single direction

For each model, we extract a single difference-in-means vector $\mathbf{r}$ via the methodology described in §2.3. We then show that this single direction is both necessary and sufficient for refusal. In §3.1, we show that ablating this direction $\hat{\mathbf{r}}$ effectively disables the model's ability to refuse harmful requests. In §3.2, we show that adding $\mathbf{r}$ to the model's activations induces refusal on harmless instructions.

3.1 Bypassing refusal via directional ablation

To bypass refusal, we perform directional ablation on the "refusal direction" $\hat{\mathbf{r}}$, ablating it from activations at all layers and all token positions. With this intervention in place, we generate model completions over JAILBREAKBENCH (Chao et al., 2024), a dataset of 100 harmful instructions.

Results are shown in Figure 1. Under no intervention, chat models refuse nearly all harmful requests, yielding high refusal and safety scores. Ablating $\hat{\mathbf{r}}$ from the model's residual stream activations, labeled as *directional ablation*, reduces refusal rates and elicits unsafe completions.

3.2 Inducing refusal via activation addition

To induce refusal, we add the difference-in-means vector $\mathbf{r}$ to activations in layer l^*, the layer that the $\mathbf{r}$ was originally extracted from. We perform this intervention at all token positions. With this intervention in place, we generate model completions over 100 randomly sampled harmless instructions from ALPACA.

Results are shown in Figure 3. Under no intervention, chat models typically do not refuse harmless instructions. Adding $\mathbf{r}$ to the model's residual stream activations, labeled as *activation addition*, results in the model refusing even harmless requests.

4 A white-box jailbreak via weight orthogonalization

In this section, we propose a novel white-box jailbreak method through *weight orthogonalization*. This technique directly modifies model weights to eliminate the representation of the refusal direction, resulting in a model that retains its original capabilities but no longer refuses harmful instructions. This new approach offers a simpler way to jailbreak open-source models compared to prior methodologies involving fine-tuning (Lermen et al., 2023; Yang et al., 2023; Zhan et al., 2023), as it does not require gradient-based optimization nor any examples of harmful completions.

4.1 Weight orthogonalization

In §2.4, we described *directional ablation* as an inference-time intervention that prevents the model from representing a direction $\hat{\mathbf{r}}$: during a forward pass, we zero out the $\hat{\mathbf{r}}$ component from every intermediate residual stream activation (Equation 4). We can equivalently implement this operation by directly modifying component weights to never write to the $\hat{\mathbf{r}}$ direction in the first place. Specifically, we can take each matrix $W_{\text{out}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{input}}}$ that writes to the residual stream, and orthogonalize its column vectors with respect to $\hat{\mathbf{r}}$:

$$W'_{\text{out}} \leftarrow W_{\text{out}} - \hat{\mathbf{r}}\hat{\mathbf{r}}^{\intercal} W_{\text{out}}. \tag{5}$$

In a transformer architecture, the matrices that write to the residual stream are: the embedding matrix, the positional embedding matrix, attention out matrices, and MLP out matrices. Orthogonalizing all of these matrices, as well as any output biases, with respect to the direction $\hat{\mathbf{r}}$ effectively prevents the model from ever writing $\hat{\mathbf{r}}$ to its residual stream.

Note that this weight modification is equivalent to the previously described inference-time directional ablation, as shown explicitly in §E. Therefore, the performance of the inference-time intervention in bypassing refusal, presented in §3.1, also exactly characterizes that of the direct weight modification.

4.2 Comparison to other jailbreaks

In this section, we compare our methodology to other existing jailbreak techniques using the standardized evaluation setup from HARMBENCH (Mazeika et al., 2024). Specifically, we generate completions over the HARMBENCH test set of 159 "standard behaviors", and then use their provided classifier model to determine the attack success rate (ASR), which is the proportion of completions classified as successfully bypassing refusal. We evaluate our weight orthogonalization method on models included in the HARMBENCH study, and report its ASR alongside those of alternative jailbreaks. For brief descriptions of each alternative jailbreak, see §F.1.

Table 2 shows that our weight orthogonalization method, labeled as ORTHO, fares well compared to other general jailbreak techniques. Across the QWEN model family, our general method is even on par with prompt-specific jailbreak techniques like GCG (Zou et al., 2023b), which optimize jailbreaks for each prompt individually.

Table 2: HARMBENCH attack success rate (ASR) across various jailbreaking methods. Our method is labeled as ORTHO. The baseline "direct response" rate with no jailbreak applied is labeled as DR. We differentiate *general* jailbreaks, which are applied across all prompts generically, from *prompt-specific* jailbreaks, which are optimized for each prompt individually. All evaluations use the model's default system prompt. We also report ASR without system prompt in blue.

	General					Prompt-specific		
Chat model	ORTHO	GCG-M	GCG-T	HUMAN	DR	GCG	AP	PAIR
LLAMA-2 7B	**22.6** (79.9)	20.0	16.8	0.1	0.0	34.5	17.0	7.5
LLAMA-2 13B	6.9 (61.0)	8.7	**13.0**	0.6	0.5	28.0	14.5	15.0
LLAMA-2 70B	4.4 (62.9)	5.5	**15.2**	0.0	0.0	36.0	15.5	7.5
QWEN 7B	**79.2** (74.8)	73.3	48.4	28.4	7.0	79.5	67.0	58.0
QWEN 14B	**84.3** (74.8)	75.5	46.0	31.5	9.5	83.5	56.0	51.5
QWEN 72B	**78.0** (79.2)	-	36.6	42.2	8.5	-	-	54.5

Note that HARMBENCH's evaluation methodology specifies that each model's default system prompt should be used during evaluation. While this approach is sensible for assessing the robustness of black-box systems, it is less applicable for white-box scenarios where attackers have full access to the model and can easily exclude the system prompt. Thus, we report ASR both with and without the system prompt.

We observe a notable difference in system prompt sensitivity across model families. For LLAMA-2 models, including the system prompt substantially reduces ASR compared to evaluation without it (e.g. 22.6% vs 79.9% for LLAMA-2 7B). In contrast, QWEN models maintain similar ASR regardless of system prompt inclusion (e.g. 79.2% vs 74.8% for QWEN 7B). While the LLAMA-2 system prompt contains explicit safety guidelines compared to the minimal QWEN system prompt, additional analysis in §F.2 suggests the discrepancy is not explained by prompt content alone, and may reflect differences in how these models respond to system-level instructions more generally.

4.3 Measuring model coherence

A reasonable concern with any new jailbreak technique is that, in addition to circumventing refusal, it may also degrade the model's overall quality (Souly et al., 2024). However, qualitatively, we observe that models maintain their coherence after undergoing weight orthogonalization. While §3.1 and §4.2 show that our method effectively bypasses refusal, in this subsection we quantitatively evaluate how the modification alters a model's general capabilities.

For each model and its orthogonalized version, we run four common language model evaluations: MMLU (Hendrycks et al., 2020), ARC (Clark et al., 2018), GSM8K (Cobbe et al., 2021), and TRUTHFULQA (Lin et al., 2021). All evaluations are run using LM Evaluation Harness (Gao et al., 2023), with settings consistent with Open LLM Leaderboard (Beeching et al., 2023).[4]

Table 3 displays that, for MMLU, ARC, and GSM8K, orthogonalized models perform similarly to baseline models. In §G.1, we show that this holds across other models in our suite, with additional evaluations of WINOGRANDE (Sakaguchi et al., 2021) and TINYHELLASWAG (Polo et al., 2024). Except for QWEN 7B and YI 34B, all evaluation metrics for orthogonalized models lie within 99% confidence intervals of original performance.

Interestingly, accuracy on TRUTHFULQA consistently drops for orthogonalized models. This phenomenon is consistent with Yang et al. (2023), where it was observed that fine-tuning away safety guardrails results in decreased accuracy on TRUTHFULQA. Examining specific questions in TRUTHFULQA reveals that the dataset veers close to the territory of refusal, with categories including "misinformation", "stereotypes", and "conspiracies", and thus it may intuitively make sense that model behavior differs meaningfully on this evaluation dataset. See §G.2 for further discussion of TRUTHFULQA performance.

In addition to standard language model evaluations, we also evaluate differences in CE loss, both on standard text corpora and model-specific generations (§G.3). These loss metrics suggest that directional ablation is more surgical than activation addition based methods (§I.1).

Table 3: Model evaluations. For each evaluation, we report the orthogonalized model's performance, followed by the baseline model's performance, followed by the absolute increase or decrease. We display the largest model from each model family. Full results are reported in §G.1.

Chat model	MMLU	ARC	GSM8K	TRUTHFULQA
GEMMA 7B	51.8 / 51.7 (+0.1)	51.7 / 51.5 (+0.2)	31.3 / 32.0 (-0.7)	44.7 / 47.1 (-2.4)
YI 34B	73.5 / 74.9 (-1.4)	65.6 / 64.9 (+0.7)	65.5 / 65.0 (+0.5)	51.9 / 55.4 (-3.5)
LLAMA-2 70B	63.1 / 63.0 (+0.1)	65.2 / 65.4 (-0.2)	54.5 / 53.0 (+1.5)	51.8 / 52.8 (-1.0)
LLAMA-3 70B	79.8 / 79.9 (-0.1)	71.5 / 71.8 (-0.3)	90.8 / 91.2 (-0.4)	59.5 / 61.8 (-2.3)
QWEN 72B	76.5 / 77.2 (-0.7)	67.2 / 67.6 (-0.4)	76.3 / 75.5 (+0.8)	55.0 / 56.4 (-1.4)

[4]As of June 2024, Open LLM Leaderboard does not use chat templates in evaluation prompts, and we follow the same practice to remain consistent. Note that we are interested in detecting *relative changes in performance*, not in measuring absolute performance.

5 Mechanistic analysis of adversarial suffixes

Safety fine-tuned chat models are vulnerable to *adversarial suffix* attacks (Zou et al., 2023b): there exist carefully constructed strings such that appending these strings to the end of a harmful instruction bypasses refusal and elicits harmful content. Effective adversarial suffixes are usually not human interpretable, and the mechanisms by which they work are not well understood. In this section, we mechanistically analyze the effect of an adversarial suffix on QWEN 1.8B CHAT.

5.1 Adversarial suffixes suppress the refusal-mediating direction

We first identify a single adversarial suffix that effectively bypasses refusal in QWEN 1.8B CHAT. The suffix is displayed in §H, along with details of its generation. To study the effect of this adversarial suffix, we sample 128 refusal-eliciting harmful instructions from JAILBREAKBENCH and the HARMBENCH test set. For each instruction, we run the model three times: first with the unedited instruction, second with the adversarial suffix appended, and third with a freshly-sampled random suffix of the same length appended. By comparing the adversarial suffix to random suffixes, we aim to control for the effect of appending any suffix at all. For each run, we cache the last token activations and visualize their cosine similarity with the refusal-mediating direction. We also compare to a baseline of 128 harmless instructions from ALPACA that do not elicit refusal. Figure 5 shows that the expression of the refusal direction is very high for harmful instructions, and remains high when a random suffix is appended. The expression of the refusal direction after appending the adversarial suffix is heavily suppressed, and closely resembles that of harmless instructions.

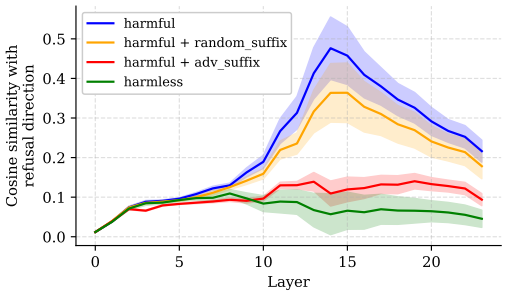
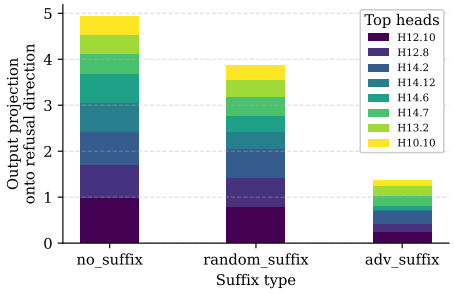


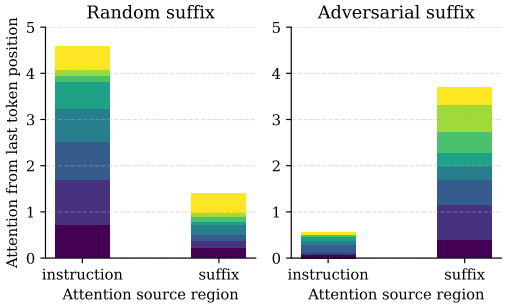
Figure 5: Cosine similarity between last token residual stream activations and refusal direction.

5.2 Adversarial suffixes hijack the attention of important heads

To further investigate how the refusal direction is suppressed, we examine the contributions of individual attention head and MLP components to the refusal direction. We quantify each component's contribution to this direction using *direct feature attribution* (DFA) (Kissane et al., 2024; Makelov et al., 2024): each component's direct contribution can be measured by projecting its output onto the refusal direction. We select the top eight attention heads with the highest DFA on harmful instructions,



(a) Attention head outputs at last token position, projected onto refusal direction.



(b) Attention from last token position to source token regions.

Figure 6: We analyze the top eight attention heads that most significantly write to the refusal direction. Figure 6(a) shows that output to the refusal direction is heavily suppressed when the adversarial suffix is appended. Figure 6(b) reveals that, compared to appending a random suffix, appending the adversarial suffix shifts attention from tokens in the instruction region to tokens in the suffix region.